COMPLEX ENGINEERING PROBLEM

TITLE

Secure Pull Request Workflow Using GitHub Branch Protection

PURAV CHAUHAN [92510103040]
JAY KANERIYA [92510103046]
RUDRA LODHARI [92510103054]

DEPARTMENT

COMPUTER ENGINEERING

ACADEMIC YEAR

2026–2027

SEMESTER

5TH

SUBJECT

OPEN SOURCE TECHNOLOGIES(01CE0526)

CLASS

5EC3

BATCH

B

COMPLEX ENGINEERING PROBLEM - REPORT

RUBRICS FOR COMPLEX ENGINEERNG PROBLEM

Sr. No.	Assessment Criteria	Excellent	Good	Satisfactory	Poor	Marks
1	Problem Analysis & Open Source Technology Selection	3 Marks: Clearly identifies the engineering problem, analyzes requirements and constraints, and selects appropriate Open Source technologies with proper justification.	2 Marks: Good problem analysis with suitable technology selection and minor gaps.	1 Mark: Basic analysis with limited justification of technology selection.	0 Marks: Inadequate problem analysis and inappropriate technology selection.	3
2	System Design, Implementation Strategy & Technical Analysis	4 Marks: Well-defined system architecture, appropriate technology stack, database, deployment strategy, version control, and comprehensive technical analysis including scalability, reliability, and security.	3 Marks: Good system design and technical discussion with minor omissions.	1–2 Marks: Basic architecture and limited technical analysis.	0 Marks: Poor or missing system design and technical analysis.	4
3	Testing, Documentation & References	3Marks: Comprehensive testing strategy, well-organized report, IEEE references, appropriate diagrams, and clear conclusions.	2 Marks: Good documentation with minor formatting or testing gaps.	1 Mark: Basic report with limited testing or references.	0 Marks: Incomplete report with poor documentation and missing references.	3
	Total					10

Sr. No.	Complex Engineering Problem Activity / Problem Title	COs	Date	R1(4)	R2(3)	R3(4)	Marks (10)	Signature
1	Secure Pull Request Workflow Using GitHub Branch Protection							

Notes / Instructions

1. CEP activity must be completed in a group of maximum 3 students.
2. Students must strictly follow the given deadlines for submission.
3. The report must be prepared in the prescribed format and submitted as a DOCX file on Google Classroom.
4. A hard copy of the report must be submitted as discussed.
5. The report should include problem analysis, design/architecture, implementation, results, and challenges faced.
6. All group members must contribute equally and be prepared for evaluation/presentation.
7. Plagiarism or copying from other groups will not be accepted.
8. Each group member must understand the complete solution and should be able to explain their contribution during evaluation.

Ms. Priyanka Mangi
Subject Faculty

COMPLEX ENGINEERING PROBLEM - REPORT

TITLE

Course Name and Course Name: OPEN SOURCE TECHNOLOGIES(01CE0526)
Co's Mapped -
SDG's Mapped -

1. Aim

To analyse, design, and demonstrate a secure pull request workflow in an open-source repository. The workflow ensures that changes are developed on feature branches, reviewed by a peer, validated by automated checks, and merged into the protected `main` branch only after the required conditions are satisfied.

2. Problem Statement

In an uncontrolled repository, a contributor may push unreviewed or defective changes directly to the main branch. Such changes can break the build, introduce security defects, remove useful history, or make it difficult to identify who approved the change. The engineering problem is to design a low-cost, auditable, and repeatable collaboration workflow that prevents direct updates to the production branch while preserving fast development on feature branches.

The proposed solution uses Git, GitHub pull requests, branch protection rules, peer review, and automated status checks. A feature branch is created from `main`; the contributor commits and pushes changes, raises a pull request, responds to review comments, and merges only after approval and successful validation.

3. Problem Analysis

a) What is the engineering problem?

The problem is to prevent unreviewed code from reaching the protected branch while allowing multiple developers to work concurrently. The workflow must enforce review, preserve traceability, detect defects before merge, and provide evidence that each change followed the approved process.

b) Why is this problem complex?

This is a complex engineering problem because it combines software configuration, version-control operations, human review, automated testing, access control, and auditability. A technically correct solution must also handle rejected reviews, new commits after approval, merge conflicts, failed checks, and emergency exceptions without weakening normal protections.

The problem involves:

- Multiple engineering concepts: version control, software quality, access control, and continuous integration.
- Analysis and decision making: choosing an appropriate branching model, review threshold, and merge strategy.
- Design constraints: limited student-team resources, repository permissions, and the requirement to retain an audit trail.
- Engineering tools: Git, GitHub, a code editor, and a CI workflow.
- Reliability and security considerations: prevention of accidental changes, reproducible validation, least-privilege access, and protection against bypassing review.

COMPLEX ENGINEERING PROBLEM - REPORT

4. Knowledge and Tools Required

Engineering concepts used

1. **Distributed version control:** Git records commits, branches, merges, and authorship locally and remotely.
2. **Branching and integration:** Feature branches isolate work until it is ready to be reviewed and integrated.
3. **Peer review and quality assurance:** Reviewers inspect the diff, discuss risks, request changes, and approve the final version.
4. **Continuous integration:** Automated checks execute on the pull request and provide a merge gate.
5. **Security and governance:** Protected branches, least privilege, signed commits, and audit records reduce the chance of unauthorized or unsafe changes.

Tools and software used

Tool	Purpose
Git	Create branches, record commits, push changes, and merge approved work.
GitHub repository	Host the source code, pull request, review discussion, checks, and merge history.
GitHub branch protection or ruleset	Require pull requests, approvals, status checks, and restrictions on direct pushes.
GitHub Actions	Run repeatable linting and test checks on pull requests.
Code editor and terminal	Modify files and execute local validation commands.
Web browser	Configure repository settings and inspect the audit trail.

5. Proposed Solution Design

5.1 Architecture diagram

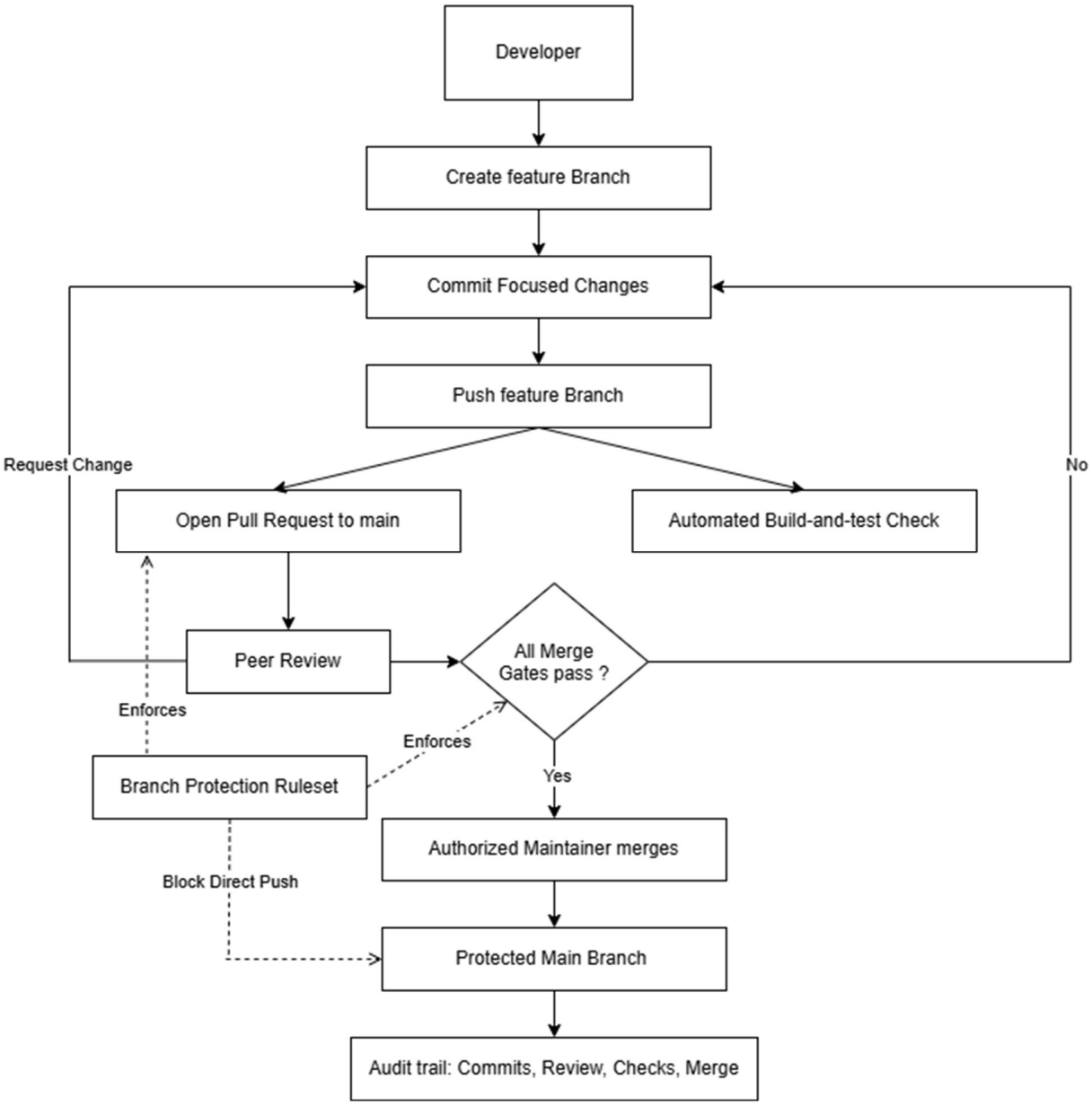


Figure 1. Secure pull request workflow from feature development to protected-branch merge.

5.2 Design decisions

The repository uses `main` as the protected integration branch. Contributors do not work directly on `main`; instead, they create branches using a descriptive naming convention such as `feature/add-validation` or `fix/input-check`. Each meaningful change is recorded as a commit with a clear message.

The pull request is the controlled integration point. It contains the purpose of the change, implementation summary, testing evidence, and any known limitations. A reviewer examines the changed files and may submit a comment, approve the pull request, or request changes. GitHub defines these review outcomes explicitly, which makes the decision visible in the pull request timeline [2].

The `main` branch is configured with the following controls:

Control	Proposed setting	Reason
Pull request before merging	Required	Prevents direct, unreviewed updates.
Required approvals	At least one peer approval	Ensures independent inspection of the change.
Dismiss stale approvals	Enabled	Requires re-review when new commits change the reviewed diff.
Approval by last contributor	Disabled for the author	Prevents a contributor from self-approving their own latest change.
Required status check	`build-and-test`	Blocks merge when automated validation fails.
Conversation resolution	Required	Ensures review concerns are addressed.
Force pushes	Disabled	Preserves branch history and prevents destructive rewrites.
Branch deletion	Disabled for `main`	Protects the integration branch.
Administrator bypass	Disabled where supported	Makes the normal workflow apply to administrators as well.

GitHub protected branches can require reviews, status checks, conversation resolution, signed commits, linear history, or deployment success before a merge [1]. A ruleset may be preferred for larger organizations because multiple rulesets can apply to the same branch and the most restrictive applicable rule is enforced [3].

5.3 Brief working

1. The contributor clones the repository and synchronizes the local `main` branch.
2. The contributor creates a feature branch from the latest `main` commit.
3. The contributor implements the change and commits it locally.
4. Local tests are executed before the branch is pushed to GitHub.
5. The contributor pushes the feature branch and opens a pull request targeting `main`.
6. GitHub starts the configured status checks and records their results.
7. A peer reviews the diff and submits comments, approval, or a request for changes.
8. The contributor addresses requested changes in additional commits and replies to the discussion.
9. The pull request becomes mergeable only when the approval, status checks, and conversation requirements are satisfied.

10. An authorized maintainer merges the pull request and records the merge commit in the repository history.

6. Implementation / Procedure

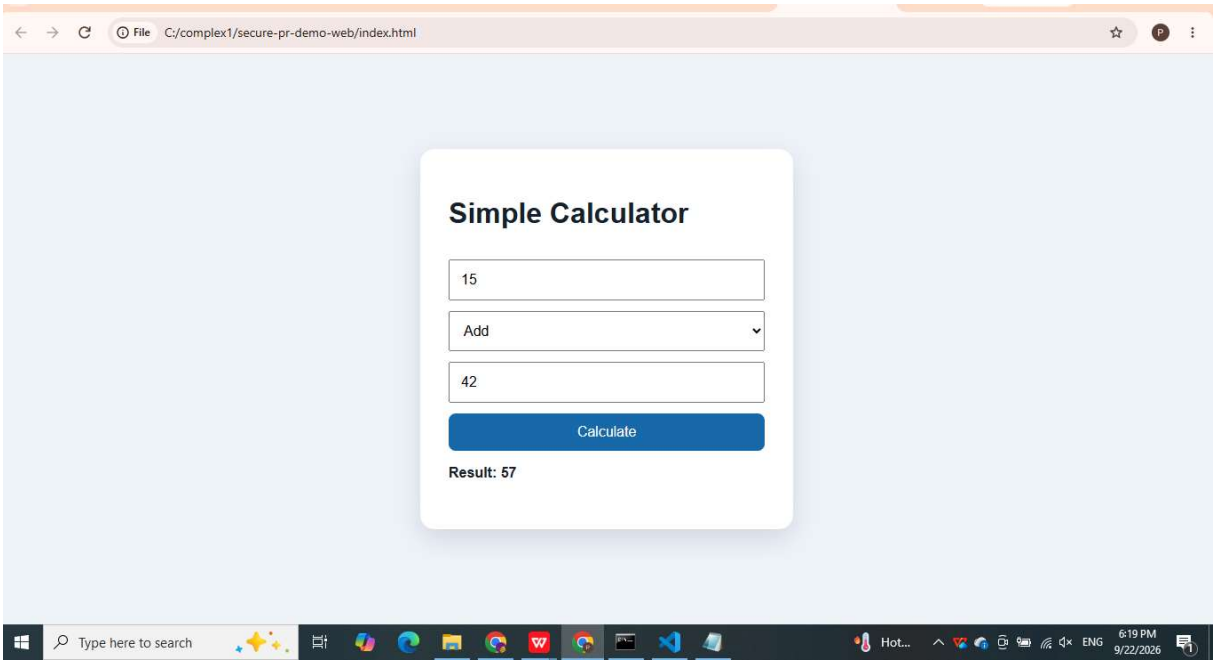
Step 1: Create the GitHub repository

1. Go to GitHub.
2. Click New repository.
3. Enter the repository name:
4. Select Public.
5. Do not select Add a README file.
6. Click Create repository.

Step 2: Open the project in your computer

You should see:
text
index.html
style.css
app.js
test.js
README.md

Open index.html in a browser by double-clicking it.

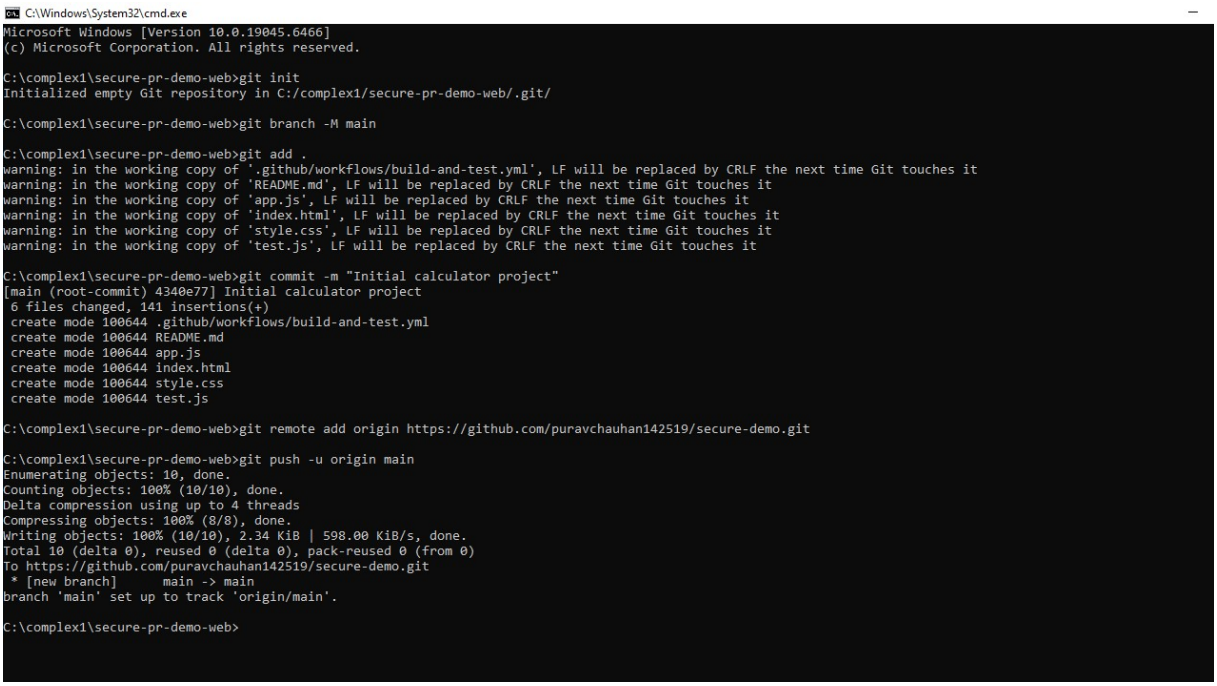


COMPLEX ENGINEERING PROBLEM - REPORT

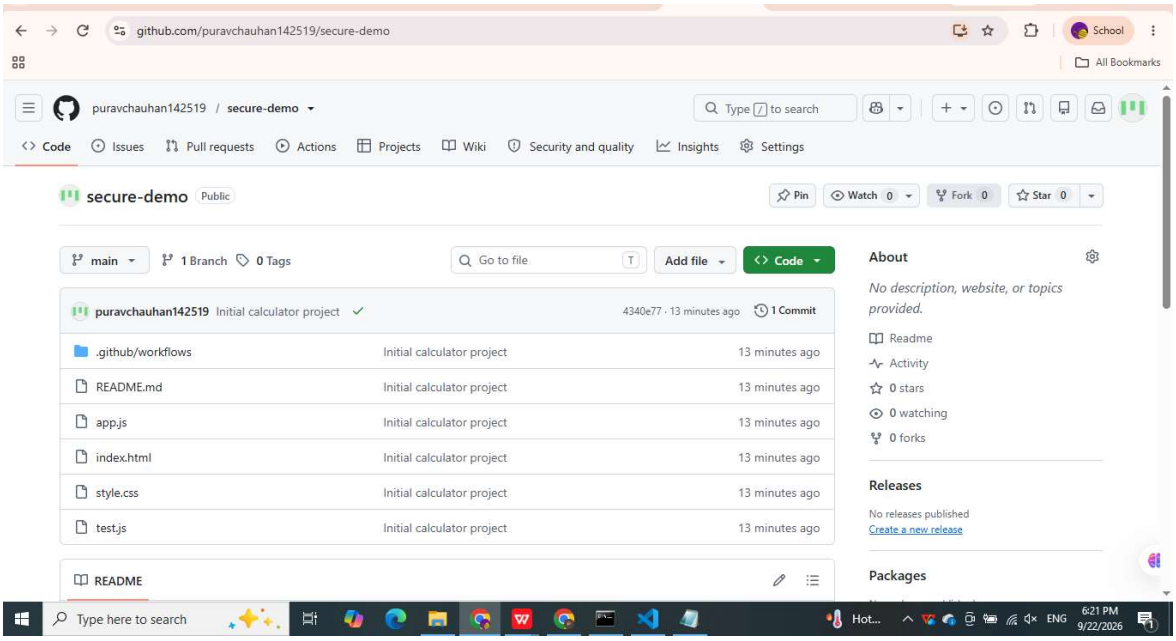
Step 3: Upload the initial project to GitHub

Open the terminal inside the project folder and run:

git init
git branch -M main
git add .
git commit -m "Initial calculator project"
git remote add origin https://github.com/<YOUR_USERNAME>/secure-pr-demo.git
git push -u origin main



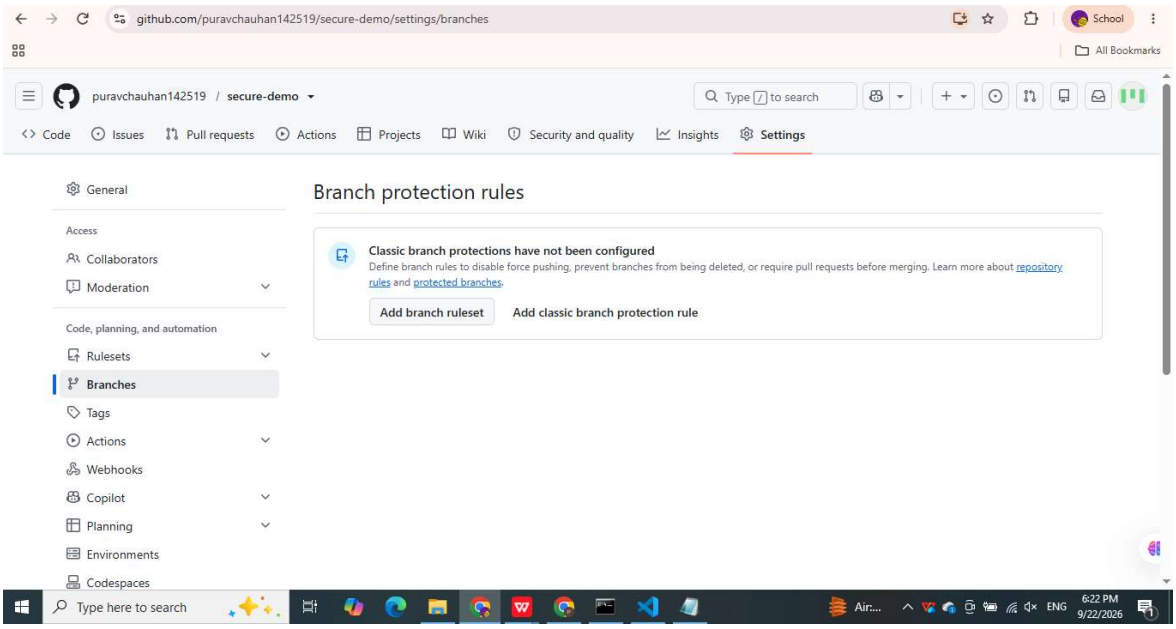
COMPLEX ENGINEERING PROBLEM - REPORT



Step 4: Configure branch protection

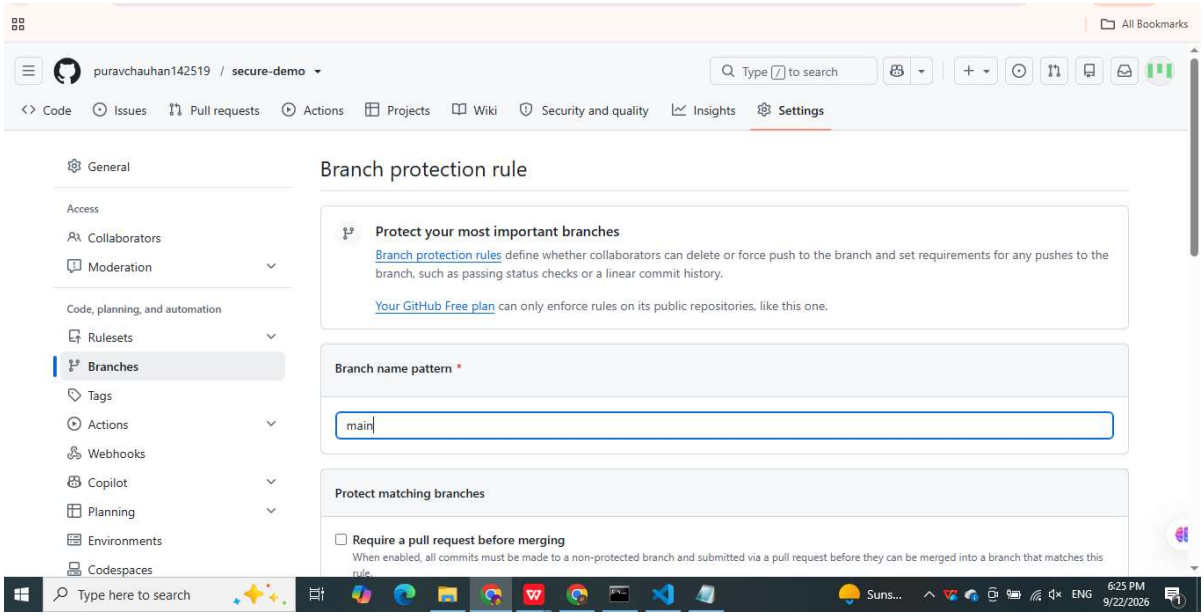
On GitHub, open:
Text

Settings → Branches → Add branch ruleset



Set the branch name or pattern to:

Name : main



Enable the following options:

- Require a pull request before merging.
- Require at least one approval.
- Dismiss stale pull request approvals when new commits are pushed.
- Require approval from someone other than the person who pushed the changes.
- Require status checks to pass before merging.
- Select the status check named:

build-and-test
Require conversation resolution before merging.

- Do not allow force pushes.
- Do not allow branch deletion.
- Click Save changes.

Step 5: Verify the JavaScript test workflow

Paste in notpad

name: Build and Test

COMPLEX ENGINEERING PROBLEM - REPORT

```
on:
  push:
    branches:
      - main
      - "feature/**"
  pull_request:
    branches:
      - main

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: Run JavaScript tests
        run: node test.js
```

```
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

C:\complex1\secure-pr-demo-web>mkdir .github
A subdirectory or file .github already exists.

C:\complex1\secure-pr-demo-web>mkdir .github\workflows
A subdirectory or file .github\workflows already exists.

C:\complex1\secure-pr-demo-web>notepad .github\workflows\build-and-test.yml

C:\complex1\secure-pr-demo-web>git switch main
M       .github/workflows/build-and-test.yml
Already on 'main'
Your branch is up to date with 'origin/main'.

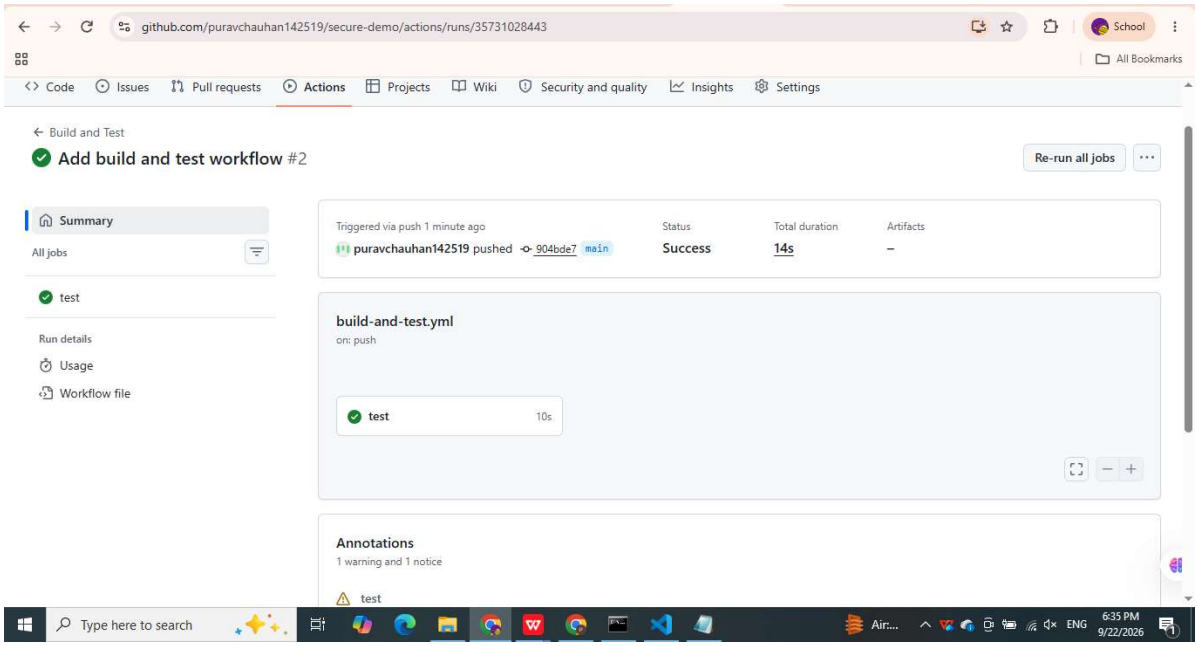
C:\complex1\secure-pr-demo-web>git add .
warning: in the working copy of '.github/workflows/build-and-test.yml', LF will be replaced by CRLF the next time Git touches it

C:\complex1\secure-pr-demo-web>git commit -m "Add build and test workflow"
[main 904bde7] Add build and test workflow
 1 file changed, 14 insertions(+), 10 deletions(-)

C:\complex1\secure-pr-demo-web>git push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 528 bytes | 528.00 KiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/puravchauhan142519/secure-demo.git
   4340e77..904bde7  main -> main

C:\complex1\secure-pr-demo-web>
```

COMPLEX ENGINEERING PROBLEM - REPORT



Step 6: Create a feature branch

Open the terminal in the project folder.

Run:

git switch main
git pull --ff-only origin main
git switch -c feature/add-division

```
   4340e77..904bde7  main -> main

C:\complex1\secure-pr-demo-web>git switch main
Already on 'main'
Your branch is up to date with 'origin/main'.

C:\complex1\secure-pr-demo-web>git pull --ff-only origin main
From https://github.com/puravchauhan142519/secure-demo
 * branch            main       -> FETCH_HEAD
Already up to date.

C:\complex1\secure-pr-demo-web>git switch -c feature/add-division
Switched to a new branch 'feature/add-division'

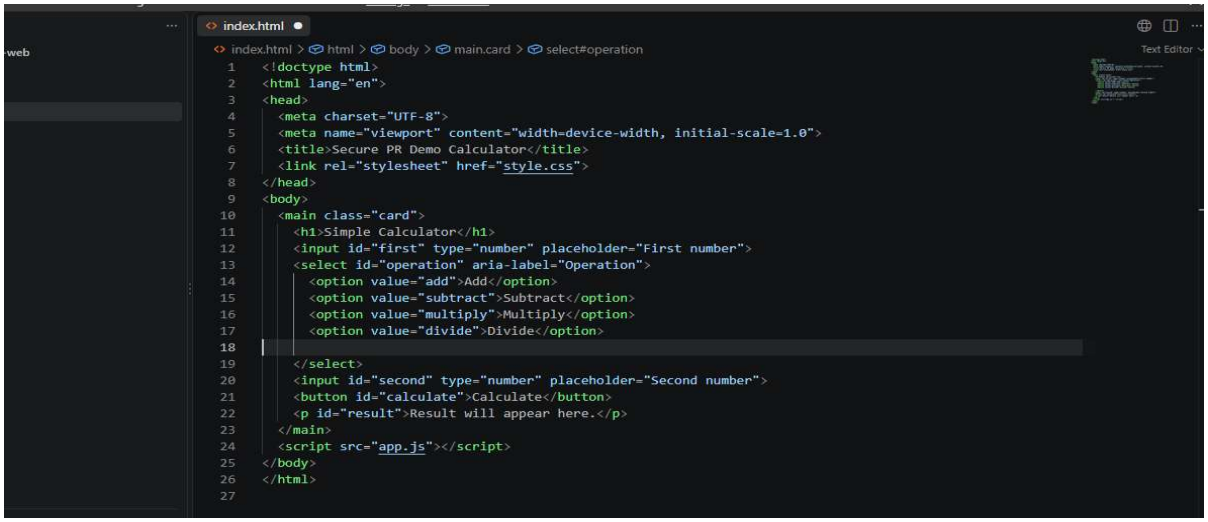
C:\complex1\secure-pr-demo-web>git branch
* feature/add-division
  main

C:\complex1\secure-pr-demo-web>
```

Step 7: Add a feature

COMPLEX ENGINEERING PROBLEM - REPORT

Open index.html.
Add this option inside the <select> element:



The operation section should look like this:

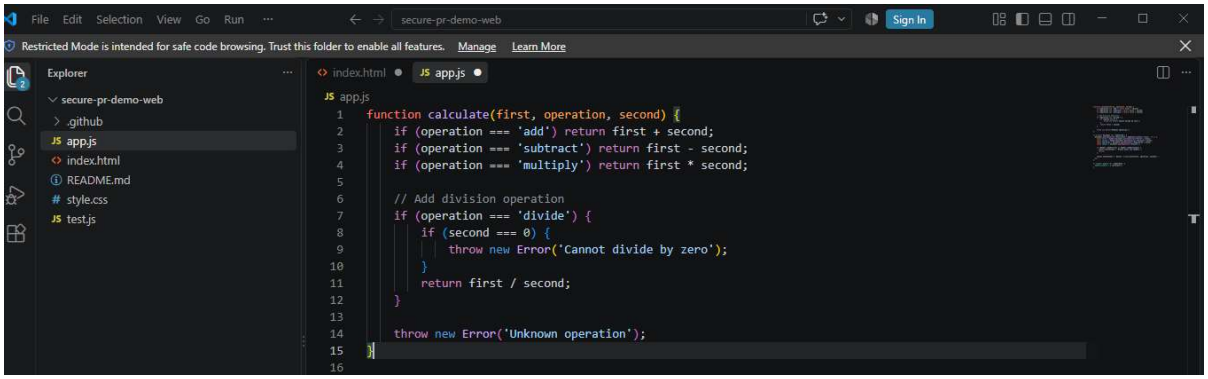
```
<select id="operation" aria-label="Operation">
  <option value="add">Add</option>
  <option value="subtract">Subtract</option>
  <option value="multiply">Multiply</option>
  <option value="divide">Divide</option>
</select>
```

Open app.js.

Add the division logic inside the calculate function:

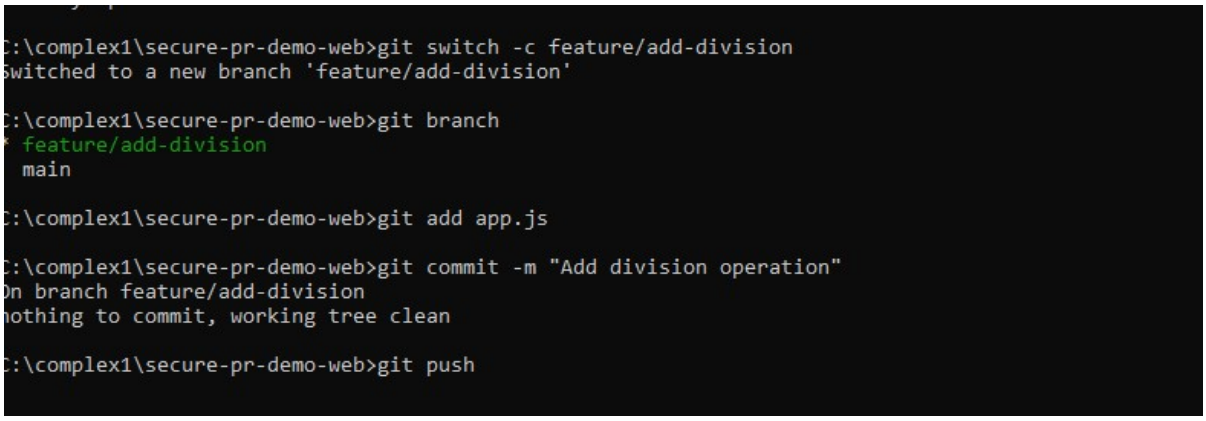
```
  if (operation === 'divide') {
    if (second === 0) {
      throw new Error('Cannot divide by zero');
    }
    return first / second;
  }
```

COMPLEX ENGINEERING PROBLEM - REPORT



Open test.js and add:
JavaScript
assert.equal(calculate(8, 'divide', 2), 4);

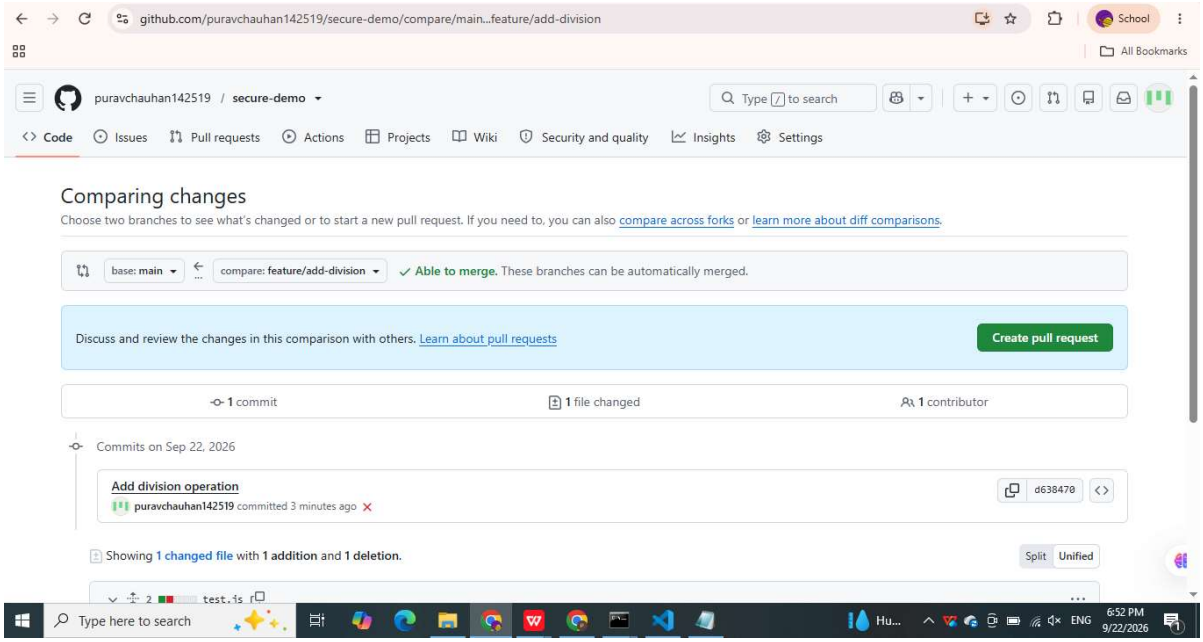
git add .
git commit -m "Add division operation"
git push



Step 8: Create the Pull Request

A pull request was created from:

COMPLEX ENGINEERING PROBLEM - REPORT



To :

Text : main

The pull request was given the title:
Text : Add division operation

The pull request description included the problem, implementation details, testing information, and review request.

The description was written as follows:

Problem:
The calculator did not support division.

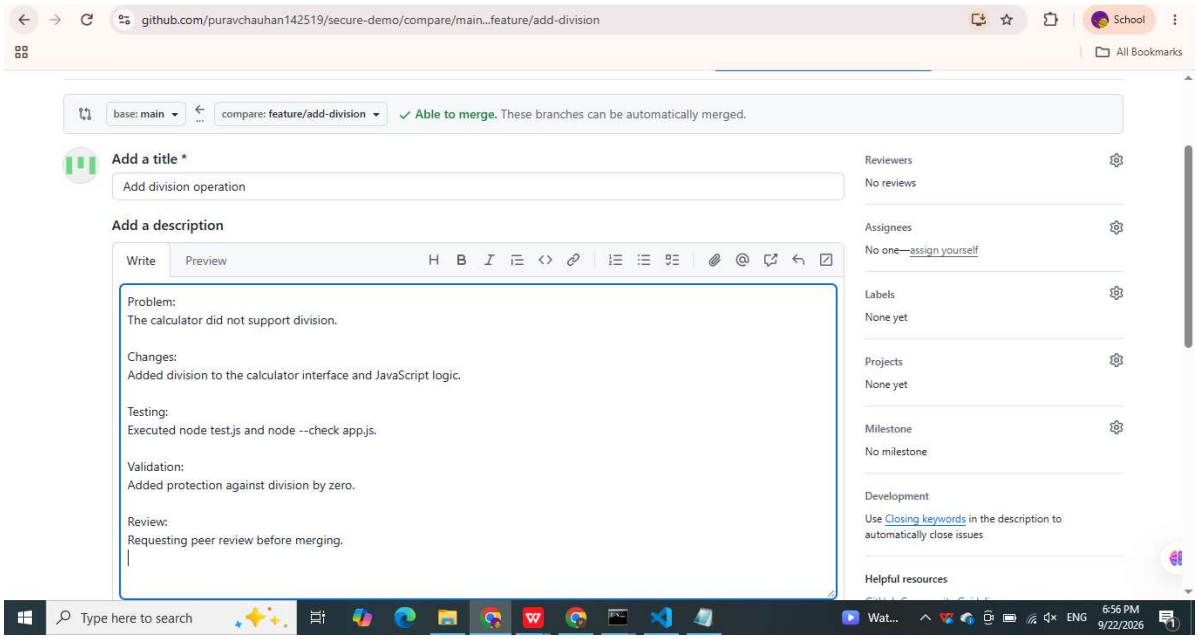
Changes:
Added division to the calculator interface and JavaScript logic.

Testing:
Executed node test.js and node --check app.js.

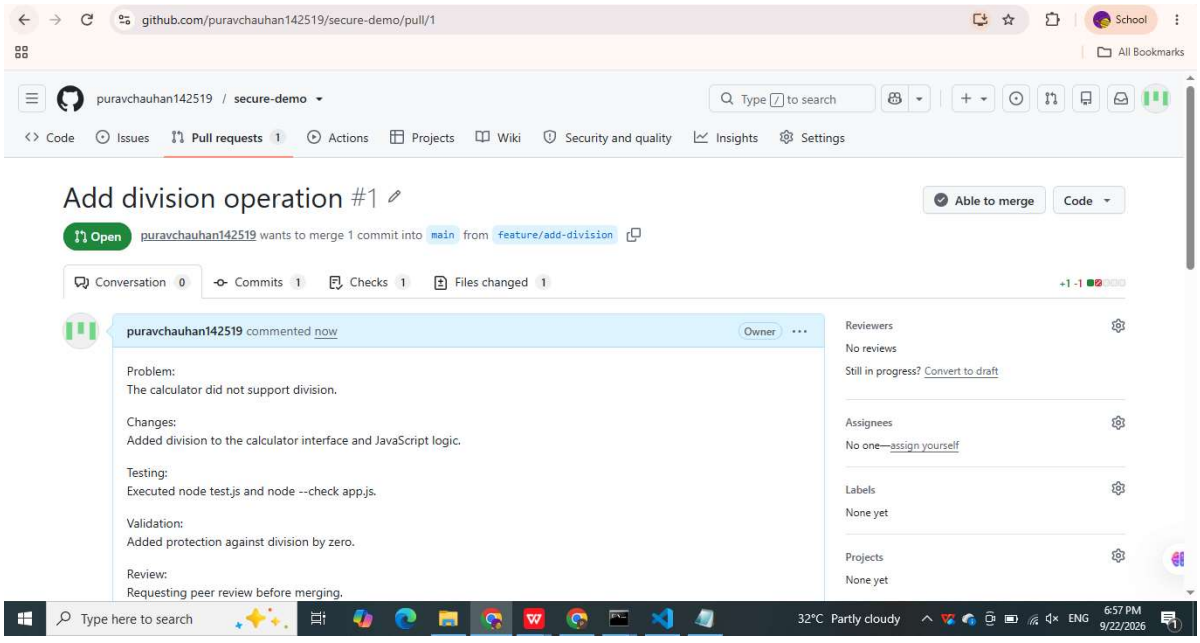
Validation:
Added protection against division by zero.

COMPLEX ENGINEERING PROBLEM - REPORT

Review:
Requesting peer review before merging.



Pull the request



Step 9: Perform Peer Review

A peer reviewer inspected the files changed in the pull request. The review focused on:

- Correctness of the division operation.
- Division-by-zero validation.
- Test coverage.
- Code readability.
- Unrelated changes.
- Possible security or reliability issues.

The reviewer added the following comment:
text
The division operation is correctly implemented. Division-by-zero validation and test cases are included. The changes are suitable for merging.
The reviewer selected Approve after completing the inspection.

Step 10: Verify the Merge Requirements

Before merging, the following conditions were verified:

- At least one peer approval was available.
- The build-and-test status check passed.
- All review conversations were resolved.
- No merge conflict was present.
- The final code contained only the intended division feature.
- The feature branch was ready to be merged into main.
- The branch protection policy prevented merging until all required conditions were satisfied.

Step 11: Merge the Pull Request

After the approval and successful automated checks, the pull request was merged into the protected main branch using the Squash and merge option.
The merge operation created a permanent record of:

- The feature branch.
- The commit.
- The pull request.
- The review comment.
- The approval.
- The automated test result.
- The final merge.

Step 12: Verify the Final Repository

After merging, the main branch was opened and checked. The final repository contained the division feature and the updated test cases.

The pull request timeline preserved the complete audit trail, including the contributor, reviewer, comments, status checks, and merge event.

7. Testing and Results

Test/Parameter Expected Result Obtained Result

7.1 Test plan

Test / parameter	Expected result	Obtained result
Direct push to `main`	Push is rejected or blocked by branch protection.	Record the observed GitHub message and attach a screenshot.
Pull request without approval	Merge button remains blocked.	Record the blocked merge state.
Pull request with a failing status check	Merge remains blocked.	Record the failed check and blocked state.
Review with requested changes	Pull request cannot be merged until the blocking review is resolved.	Record reviewer comment and author response.
New commit after approval	Approval becomes stale or the latest push requires re-approval according to policy.	Record the observed behavior.
Pull request with approval and passing checks	Merge becomes available to an authorized maintainer.	Record the merged state and timestamp.
Post-merge traceability	Repository shows the branch, commits, reviewer, checks, and merge event.	Record repository and pull request URLs.

7.2 Expected result

The workflow should prevent direct changes to `main`, make review mandatory, and block merges when validation fails. It should also leave a traceable record of the author, commits, reviewer decisions, status checks, and final merge. These controls reduce accidental integration and improve the reliability of collaboration.

7.3 Evidence to attach

The following evidence must be replaced with the group's actual links and screenshots before submission. No repository activity should be reported as completed unless it was actually performed.

Evidence item	Value to provide
Repository link	`https://github.com/<OWNER>/<REPOSITORY>`
Feature branch	`<feature-branch-name>`
Commit link	`<commit-URL>`
Pull request link	`<pull-request-URL>`

Review comments	Screenshot or URL showing peer feedback.
Approval evidence	Screenshot or URL showing the peer's **Approved** review.
Status-check evidence	Screenshot showing `build-and-test` passed.
Merge evidence	Screenshot or URL showing the pull request merged into `main`.
Branch protection evidence	Screenshot of the active `main` protection settings.

Suggested figure captions: Figure 2, Branch protection settings; Figure 3, feature-branch commit; Figure 4, pull request and review comment; Figure 5, successful status check; Figure 6, merged pull request.

8. Challenges Faced and Solutions

Challenge	Solution
A contributor may accidentally push directly to `main`.	Enable protected-branch rules that require a pull request and disable direct updates.
A review may be approved before the final code is pushed.	Enable stale-approval dismissal or require approval of the most recent reviewable push.
Automated checks may be unreliable or ambiguous.	Use stable, unique job names and make the required check match the workflow job name.
Review comments may remain unresolved.	Require conversation resolution and verify every blocking thread before merge.
A pull request may contain unrelated edits.	Keep commits focused, inspect `git diff`, and use a pull request checklist.
The branch may conflict with new changes in `main`.	Update the feature branch, resolve conflicts locally, rerun tests, and obtain re-approval when required.
Administrators may bypass controls.	Disable bypass where the repository plan supports it and document any exceptional procedure.
Team members may lack permission to review or configure rules.	Assign least-privilege roles and perform configuration using an authorized repository maintainer.

9. Conclusion

The secure pull request workflow solves the problem of uncontrolled changes entering the main integration branch. It separates development from integration through feature branches and uses GitHub pull requests as a controlled review boundary. Branch protection ensures that approval, automated validation, and conversation resolution are completed before merging. The workflow also preserves a useful audit trail containing commits, comments, status checks, reviewers, and merge information. As a result, the team gains better code quality, lower integration risk, and clearer responsibility for changes. The design is lightweight enough for a student project and can be extended with code owners, signed commits, deployment checks, and organization-level rulesets.

10. SDG Mapping and Justification

SDG 9: Industry, Innovation and Infrastructure

The workflow supports resilient digital infrastructure by applying an open-source toolchain to software development. Automated checks, controlled integration, and reproducible procedures improve the reliability of the software development infrastructure.

SDG 16: Peace, Justice and Strong Institutions

The workflow promotes transparent and accountable technical decision making. Review records, approval requirements, access restrictions, and an auditable merge history make it possible to understand how a change was evaluated and who authorized it.

11. References

[1] GitHub Docs, "About protected branches," GitHub, accessed Sep. 22, 2026. [Online]. Available: https://docs.github.com/repositories/configuring-branches-and-merges-in-your-repository/defining-the-mergeability-of-pull-requests/about-protected-branches
[2] GitHub Docs, "Pull request reviews," GitHub, accessed Sep. 22, 2026. [Online]. Available: https://docs.github.com/pull-requests/reference/pull-request-reviews
[3] GitHub Docs, "About rulesets," GitHub, accessed Sep. 22, 2026. [Online]. Available: https://docs.github.com/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/about-rulesets
[4] GitHub Docs, "Reviewing proposed changes in a pull request," GitHub, accessed Sep. 22, 2026. [Online]. Available: https://docs.github.com/en/pull-requests/how-tos/review-pull-requests/reviewing-proposed-changes-in-a-pull-request

COMPLEX ENGINEERING PROBLEM - REPORT

Assessment Mapping

Rubric criterion	Where demonstrated
Problem analysis and open-source technology selection	Sections 2–4 explain the problem, constraints, and rationale for Git, GitHub, and GitHub Actions.
System design, implementation strategy, and technical analysis	Sections 5–6 provide the architecture, branch policy, workflow, commands, CI configuration, security, scalability, and reliability considerations.
Testing, documentation, and references	Sections 7–11 provide the test plan, evidence checklist, challenges, conclusion, SDG mapping, and IEEE-style references.

Group Activity Record

Sr. No.	Complex Engineering Problem Activity / Problem Title	COs	Date	R1 (4)	R2 (3)	R3 (4)	Marks (10)	Signature
1	Secure Pull Request Workflow Using GitHub Branch Protection	CO1, CO2, CO3	`<DATE>`					

Faculty: Ms. Priyanka Mangi
Subject: Open Source Technologies (01CE0526)

Submission note: *Replace all angle-bracket placeholders with the group's actual details. Insert the requested screenshots and URLs in Section 7.3 after executing the workflow in the selected repository.*

[1]:https://docs.github.com/repositories/configuring-branches-and-merges-in-your-repository/defining-the-mergeability-of-pull-requests/about-protected-branches "About protected branches"
[2]:https://docs.github.com/pull-requests/reference/pull-request-reviews "Pull request reviews"
[3]:https://docs.github.com/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/about-rulesets "About rulesets"
[4]:https://docs.github.com/en/pull-requests/how-tos/review-pull-requests/reviewing-proposed-changes-in-a-pull-request "Reviewing proposed changes in a pull request"